SOFTWARE QUALITY ENGINEERING

Assignment 2 – Student Course Registration System

Black-Box Testing, White-Box Testing, Defect Analysis and Continuous Integration

Course: Software Quality Engineering (SE3002)

Project: Student Course Registration System

GitHub Repository: https://github.com/hannyx-dev/SQE-Assignment-2

Submitted by: Hanniya Ahmad (24P-3014) & Hamna Arshad (24P-3050)

1. Introduction

The purpose of this assignment was to evaluate the quality and correctness of an existing Student Course Registration System through systematic software testing techniques.

The assignment focused on identifying defects, designing effective test cases, measuring code coverage, performing black-box and white-box testing, and using GitHub for collaborative development and continuous integration.

The existing implementation was evaluated against its functional requirements. The objective was not to develop an entirely new registration system, but to examine the supplied implementation, identify incorrect behavior, document defects, and verify corrections through regression testing.

Both black-box and white-box testing techniques were applied. Black-box testing focused on the externally observable behavior of the system, while white-box testing examined the internal program structure, control flow, execution paths, and logical conditions.

GitHub was used for repository management, version control, collaboration, defect tracking, pull requests, peer review, and automated testing through GitHub Actions.

2. Project Scope and Testing Objectives

The testing activities covered:

1. Equivalence Class Partitioning (ECP).
2. Boundary Value Analysis (BVA).
3. Decision Table Testing.
4. Test case optimization and duplicate analysis.
5. Control Flow Graph (CFG) construction.
6. Cyclomatic Complexity calculation.
7. Independent path identification.
8. Modified Condition/Decision Coverage (MC/DC).
9. Statement and branch coverage measurement.

10. Defect identification, documentation, and correction.
11. GitHub collaboration and continuous integration.

The main objectives were to:

- Identify incorrect behavior in the registration system.
- Design test cases covering valid, invalid, and boundary inputs.
- Examine decision-making logic and independent execution paths.
- Measure testing effectiveness using code coverage.
- Verify that identified defects were corrected.
- Maintain a collaborative development workflow using GitHub.
- Automate regression testing through GitHub Actions.

3. Black-Box Testing

Black-box testing was performed without relying on the internal implementation details of the functions. Test cases were designed using the functional requirements and expected system behavior.

Hamna was responsible for the black-box testing activities, including test case design, automated test implementation, and duplicate test analysis.

3.1 Equivalence Class Partitioning (ECP)

Equivalence Class Partitioning divides the input domain into groups of values expected to produce similar behavior. Instead of testing every possible input, representative values are selected from each equivalence class.

The ECP test cases covered:

- Valid and invalid credit-hour values.
- Outstanding dues.
- Prerequisite completion.
- Course capacity and availability.
- Duplicate course registration.
- Valid and invalid registration conditions.

A total of 15 ECP test cases were designed.

The test cases included valid inputs, invalid inputs, and representative values from different equivalence classes. Invalid inputs were expected to produce the appropriate rejection or exception, while valid inputs were expected to produce the correct registration decision.

The test cases are documented in:

docs/A1_ECP_Test_Cases.xlsx

3.2 Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on values at, immediately below, and immediately above important boundaries. Errors frequently occur at these limits because of incorrect comparison operators or validation conditions.

The BVA test cases examined:

- Minimum and maximum credit-hour limits.
- The maximum permitted credit total of 18 credit hours.
- Course credit values.
- Outstanding dues around zero.
- Course capacity and enrollment boundaries.

A total of 22 BVA test cases were designed.

For example, the credit-hour limit was examined using values that tested whether the system correctly accepted a total of exactly 18 credit hours and rejected totals exceeding 18.

Course capacity was also tested at the boundary where enrollment equaled capacity. A course with an available seat should permit registration, while a full course should place an eligible student on the waitlist.

The test cases are documented in:

docs/A2_BVA_Test_Cases.xlsx

3.3 Decision Table Testing

Decision Table Testing was used to examine combinations of conditions that influence registration decisions.

The important conditions included:

- Whether the student has outstanding dues.
- Whether the student is already registered.
- Whether prerequisites are satisfied.
- Whether the credit-hour limit is exceeded.
- Whether the course has available seats.

The expected outcomes included:

- DUES_BLOCK
- ALREADY_REGISTERED
- PREREQUISITES_MISSING
- CREDIT_LIMIT
- WAITLISTED
- REGISTERED

The decision table also examined combinations of multiple invalid conditions to verify that the system applied the appropriate priority when more than one condition was violated.

For example, when a student had outstanding dues and was already registered in the course, the expected result was DUES_BLOCK, according to the defined decision priority.

Ten decision-table scenarios were documented in the decision-table workbook. Decision-table-related cases were also included in the automated black-box suite.

The test cases are documented in:

docs/A3_Decision_Table_Test_Cases.xlsx

3.4 Black-Box Automated Testing

The black-box test cases were implemented using Python's pytest framework.

The test suite is located in:

tests/test_blackbox.py

It contains 42 test cases covering ECP, BVA, decision-table scenarios, and registration-state behavior.

The tests were initially executed against the supplied implementation to identify incorrect behavior. Initial testing exposed multiple failures, demonstrating that several functional requirements were not being satisfied.

After the identified implementation problems were corrected, the black-box suite was executed again.

Final black-box result:

- Total tests: 42
- Passed: 42
- Failed: 0
- Statement coverage: 98%

The remaining uncovered statement in this intermediate black-box run was subsequently covered by the combined black-box and white-box test suite.

3.5 Test Case Optimization – A5

Test case optimization was performed to identify exact duplicates and similar test cases across ECP, BVA, and Decision Table Testing.

The objective was to reduce unnecessary repetition while preserving meaningful testing coverage.

The analysis distinguished between:

Exact duplicates: Test cases with equivalent input conditions, execution behavior, and expected outcomes.

Similar but not exact cases: Test cases that examined related requirements but differed in inputs, conditions, setup, or expected behavior.

Seven exact duplicate groups were identified and documented. Additional similar cases were recorded separately to explain why they should not automatically be treated as duplicates.

The analysis preserved the traceability of the original test cases and their associated requirements.

The results are documented in:

docs/A5_Duplicate_Analysis.xlsx

4. White-Box Testing

White-box testing examined the internal structure and control flow of the registration system.

Hanniya was responsible for the white-box analysis, including the Control Flow Graph, cyclomatic complexity, independent paths, MC/DC analysis, and white-box automated tests.

4.1 Control Flow Graph (CFG)

A Control Flow Graph represents the execution flow of a program through nodes and directed edges.

The registration decision logic was analyzed and represented using a Control Flow Graph.

The graph contains:

- Nodes (N): 17
- Edges (E): 23
- Decision nodes (D): 7

The CFG is included in:

docs/cfg.png

4.2 Cyclomatic Complexity

Cyclomatic Complexity measures the number of linearly independent paths through a program.

It was calculated using:

V(G) = E − N + 2

Where:

- E = Number of edges
- N = Number of nodes

Substituting the CFG values:

V(G) = 23 − 17 + 2

V(G) = 8

The complexity was also verified using the decision-node formula:

V(G) = D + 1

V(G) = 7 + 1

V(G) = 8

Both methods produced the same result.

Therefore, the registration logic has a cyclomatic complexity of 8, indicating that eight linearly independent paths are required for basis-path testing.

4.3 Independent Paths

Based on the cyclomatic complexity, eight independent paths were identified.

Path	Execution scenario
P1	Invalid negative outstanding dues
P2	Invalid course capacity or enrollment values
P3	Valid input followed by outstanding-dues rejection
P4	No dues block, but duplicate registration detected
P5	No duplicate registration, but prerequisites are missing
P6	Prerequisites satisfied, but credit-hour limit exceeded
P7	Earlier rejection conditions are false and the course is full, resulting in waitlisting
P8	All validation conditions are satisfied and registration succeeds

These paths were used to design white-box test cases that exercise the different execution routes through the registration logic.

The waitlisting path was particularly important because the original implementation contained a capacity-validation problem that prevented the expected waitlisting behavior. The implementation was subsequently corrected and the path was verified through regression testing.

4.4 MC/DC Analysis

Modified Condition/Decision Coverage (MC/DC) was applied to examine the individual effect of atomic conditions on a compound decision.

The analyzed decision was:

if not 0 <= course.enrolled <= course.capacity or course.capacity < 1:

The atomic conditions were:

- C1: 0 <= course.enrolled
- C2: course.enrolled <= course.capacity

- C3: course.capacity < 1

The decision can be represented as:

D = NOT (C1 AND C2) OR C3

MC/DC Test Cases

Test	Enrolled	Capacity	C1	C2	C3	Decision
T1	0	1	T	T	F	F
T2	-1	1	F	N/A	N/A	T
T3	2	1	T	F	N/A	T
T4	0	0	T	T	T	T

Independent Condition Pairs

- **C1:** T1 and T2 demonstrate that changing the lower-bound condition changes the decision.
- **C2:** T1 and T3 demonstrate that changing the enrollment-versus-capacity condition changes the decision.
- **C3:** T1 and T4 demonstrate that changing the minimum-capacity condition changes the decision.

The truth table accounts for short-circuit evaluation. Conditions that are not evaluated because of an earlier decisive result are marked N/A.

The analysis demonstrates the independent effect of all three atomic conditions.

MC/DC was analyzed manually using the truth table and independent condition pairs. It was not treated as a metric automatically measured by pytest-cov.

4.5 White-Box Automated Testing

White-box tests were implemented using pytest and are located in:

tests/test_whitebox.py

The suite contains 27 tests covering:

- The eight independent execution paths.
- MC/DC test scenarios.

- Helper functions.
- Credit-hour boundary conditions.
- Prerequisite validation.
- Registration decisions.
- Course waitlisting.
- Course withdrawal.
- Outstanding-dues validation.
- Registration-state behavior.

During initial testing, several defects were identified, including incorrect credit-limit handling, incomplete prerequisite validation, incorrect waitlisting behavior, failure to block positive outstanding dues, and incorrect enrollment updates after withdrawal.

The implementation was corrected, and the affected paths and conditions were tested again.

5. Defect Identification and Correction

Testing exposed multiple functional defects in the original implementation.

The defects were documented using GitHub Issues to provide traceability and maintain a record of the observed problems.

The main defect categories were:

Defect	**Expected behavior**	**Observed problem**
Credit-hour limit	Exactly 18 credit hours should be accepted	Exactly 18 credit hours were rejected
Outstanding dues	Positive outstanding dues should block registration	Registration was allowed despite positive dues
Prerequisites	All required prerequisites must be completed	Registration was allowed when some prerequisites were missing
Course capacity	A full course should result in waitlisting	Registration was incorrectly accepted

Withdrawal	Withdrawing a course should decrement enrollment	Enrollment count did not decrease
Registration state	Rejected registration should not modify student state	Student state changed despite rejection

The GitHub Issues include:

- #2 and #8 – Credit-hour limit defect.
- #3 and #10 – Partial prerequisite completion defect.
- #4 and #9 – Outstanding-dues defect.
- #5 and #11 – Full-course waitlisting defect.
- #6 and #12 – Course enrollment count after withdrawal.
- #13 – Student state changes when registration should be rejected.

Some issues describe the same underlying defect from different testing observations or contributors. Therefore, the issue count should not be interpreted as the number of completely independent defects.

The final automated regression tests verified the corrected behavior.

The GitHub Issues retained the original defect descriptions and testing evidence.

6. Test Execution and Coverage Results

Testing was performed in stages to evaluate the original implementation and verify the corrected system.

6.1 Initial Black-Box Testing

The initial black-box execution produced the following results:

Metric	**Result**
Total tests	42
Passed	29
Failed	13
Statement coverage	86%

The failures exposed problems in credit-hour validation, outstanding-dues handling, prerequisite validation, waitlisting, registration-state integrity, and withdrawal behavior.

6.2 Corrected Black-Box Testing

After the implementation corrections, the black-box test suite was executed again.

Metric	Result
Total tests	42
Passed	42
Failed	0
Statement coverage	98%

All black-box test cases passed.

6.3 Final Combined Testing

The black-box and white-box test suites were then executed together.

The command used was:

python -m pytest tests -v

The result was:

- Total tests: 69
- Passed: 69
- Failed: 0

The final coverage command was:

python -m pytest tests --cov=registration --cov-branch --cov-report=term-missing

The final output was:

File	Statements	Missed	Branches	Partial Branches	Coverage
registration.py	58	0	26	0	100%
TOTAL	**58**	**0**	**26**	**0**	**100%**

The final combined execution achieved:

- 100% statement coverage.
- 100% branch coverage.
- Zero missed statements.

- Zero partial branches.
- 69 successful tests out of 69.

The final results demonstrate that every measured statement and branch in the analyzed registration module was covered by the combined test suite.

Statement coverage, branch coverage, and MC/DC measure different aspects of testing. Although automated coverage reached 100% for statements and branches, MC/DC was separately evaluated through the documented truth table and independent condition pairs.

7. GitHub Collaboration and Continuous Integration

GitHub was used to manage the project, coordinate contributions, track defects, and automate test execution.

Repository:

https://github.com/hannyx-dev/SQE-Assignment-2

7.1 Repository Creation, Setup and Structure

Hanniya created and initialized the GitHub repository and was responsible for setting up the initial project structure and version-control workflow.

This included establishing the repository, organizing the project files, and preparing the environment for collaborative development.

The repository contains the following main components:

SQE-Assignment-2/

|

├── .github/

| └── workflows/

| ├── blackbox-tests.yml

| └── tests.yml

|

├── docs/

| ├── A1_ECP_Test_Cases.xlsx

| ├── A2_BVA_Test_Cases.xlsx

```
|   ├── A3_Decision_Table_Test_Cases.xlsx
|   ├── A5_Duplicate_Analysis.xlsx
|   ├── White_Box_Analysis.docx
|   └── cfg.png
|
├── tests/
|   ├── test_blackbox.py
|   └── test_whitebox.py
|
├── registration.py
└── README.md
```

The repository also contains the assignment's supporting project files and testing artifacts.

7.2 Branching Strategy

Separate branches were used to organize contributions and avoid making all changes directly on the main branch.

Hanniya worked on the white-box testing and analysis activities, including the white-box test suite and associated CI workflow.

Hamna worked on black-box test design, implementation, duplicate analysis, and the black-box CI workflow.

Changes were integrated into the main branch through pull requests.

7.3 Pull Requests and Peer Review

The project used pull requests to integrate contributions and support collaborative review.

The following pull requests were merged:

- **PR #7:** White-box testing and related workflow contributions by Hanniya.
- **PR #15:** Black-box testing and related contributions by Hamna.

Both team members participated in the pull-request and peer-review process to support collaborative development and integration.

7.4 GitHub Issues

GitHub Issues were used to document defects discovered during testing.

Each issue provided a record of the defect, expected behavior, observed behavior, and relevant testing information.

This made it easier to track problems and associate testing observations with implementation corrections.

7.5 GitHub Actions

Two GitHub Actions workflows were configured:

1. blackbox-tests.yml
2. tests.yml

These workflows automate test execution when relevant repository events occur, including pushes and pull requests.

The black-box workflow runs the black-box test suite, while the main testing workflow executes the Python tests and coverage measurement.

Automated checks provide an additional verification step whenever changes are proposed or integrated.

The workflows help detect regressions and ensure that the test suites are executed consistently.

8. Individual Contributions

The assignment was completed collaboratively, with responsibilities divided between Hanniya and Hamna.

8.1 Hanniya

Hanniya was responsible for:

- Creating and setting up the GitHub repository.
- Establishing the initial repository structure and version-control workflow.
- Organizing the project environment for collaborative development.
- White-box testing and analysis.
- Control Flow Graph construction.
- Identification of nodes, edges, and decision nodes.

- Cyclomatic Complexity calculation.
- Identification of eight independent paths.
- MC/DC truth table and condition-pair analysis.
- Development of tests/test_whitebox.py.
- Documentation of white-box testing results.
- Identification and documentation of white-box defects through GitHub Issues.
- White-box-related GitHub branch and workflow contributions.
- Creating and merging PR #7.
- Reviewing Hamna's contributions through PR #15.
- Participating in final regression testing and report preparation.

8.2 Hamna

Hamna was responsible for:

- Equivalence Class Partitioning test design.
- Boundary Value Analysis test design.
- Decision Table test design.
- Development of tests/test_blackbox.py.
- Test case optimization and duplicate analysis.
- Preparation of the ECP, BVA, Decision Table, and A5 workbooks.
- Identification and documentation of black-box defects through GitHub Issues.
- Black-box-related implementation corrections.
- Black-box GitHub branch and workflow contributions.
- Creating and merging PR #15.
- Participating in collaborative testing, peer review, and project integration.

Both Hanniya and Hamna contributed to the completion of the assignment through their respective testing activities and the integration of their work.

9. Challenges and Lessons Learned

Several challenges were encountered during the assignment.

9.1 Identifying Boundary-Related Defects

The credit-hour validation logic demonstrated how a small difference in a comparison operator can result in incorrect behavior at an exact boundary.

Testing values around the maximum credit-hour limit helped expose this issue.

9.2 Testing Combined Conditions

Registration decisions depend on multiple conditions. Testing individual conditions alone was insufficient to verify the correct priority when several conditions were violated simultaneously.

Decision Table Testing helped identify these combinations and verify the expected outcomes.

9.3 Understanding Control Flow

Constructing the CFG and calculating cyclomatic complexity provided a clearer understanding of the internal registration logic.

Identifying independent paths helped ensure that different execution routes were tested rather than relying only on ordinary successful inputs.

9.4 Distinguishing Coverage Metrics

The assignment demonstrated that statement coverage, branch coverage, and MC/DC measure different aspects of testing.

A high statement coverage percentage does not necessarily mean every branch or independent condition has been adequately tested.

9.5 Collaborative Development

Using separate branches, pull requests, and GitHub Issues helped organize the work and maintain a history of changes.

GitHub Actions also demonstrated how automated testing can support continuous integration and help identify regressions.

10. Conclusion

This assignment provided practical experience in applying software quality assurance techniques to an existing Student Course Registration System.

Black-box testing was performed using Equivalence Class Partitioning, Boundary Value Analysis, Decision Table Testing, and test case optimization. White-box testing involved Control Flow

Graph construction, Cyclomatic Complexity calculation, independent path testing, and MC/DC analysis.

The testing process identified several functional defects, including incorrect credit-hour validation, outstanding-dues handling, prerequisite validation, course waitlisting, registration-state modification, and withdrawal enrollment updates.

After the identified implementation corrections, the final combined test execution produced **69 passing tests out of 69**, with **100% statement coverage and 100% branch coverage** for the analyzed registration module.

GitHub supported collaboration through repository creation and setup, branches, issues, pull requests, peer review, and automated GitHub Actions workflows.

Overall, the assignment demonstrated the importance of systematic test design, defect tracking, code coverage analysis, regression testing, and collaborative software development in improving software quality.

11. Project Artifacts and Evidence

The following artifacts form the supporting evidence for the assignment:

Artifact	**Description**
registration.py	Student Course Registration System implementation
tests/test_blackbox.py	Black-box automated test suite
tests/test_whitebox.py	White-box automated test suite
docs/A1_ECP_Test_Cases.xlsx	Equivalence Class Partitioning test cases
docs/A2_BVA_Test_Cases.xlsx	Boundary Value Analysis test cases
docs/A3_Decision_Table_Test_Cases.xlsx	Decision Table test cases
docs/A5_Duplicate_Analysis.xlsx	Exact and similar duplicate test analysis
docs/White_Box_Analysis.docx	CFG, cyclomatic complexity, independent paths, MC/DC, and white-box results
docs/cfg.png	Control Flow Graph
.github/workflows/blackbox-tests.yml	Automated black-box testing workflow

.github/workflows/tests.yml	Automated Python testing and coverage workflow
GitHub Issues	Defect identification and tracking
PR #7 and PR #15	Contribution integration and review evidence
testing_report.pdf	Consolidated assignment report

Final Testing Status: 69/69 tests passed; 100% statement coverage; 100% branch coverage.

GitHub Repository: https://github.com/hannyx-dev/SQE-Assignment-2